COMPUTER GRAPHICS

Assignment 2 – Answers (Simple Notes)

Q1. 2D Geometrical Transformation and Basic 2D Transformations

A **geometrical transformation** changes the **position, size or orientation** of an object by changing the coordinates of its points. In **2D**, every point (x, y) is mapped to a new point (x′, y′).

- Used to move, resize, turn and flip objects in animation, games, CAD and display systems.
- An object is transformed by transforming its **end points (vertices)**; lines are then redrawn between them.

Basic 2D transformations

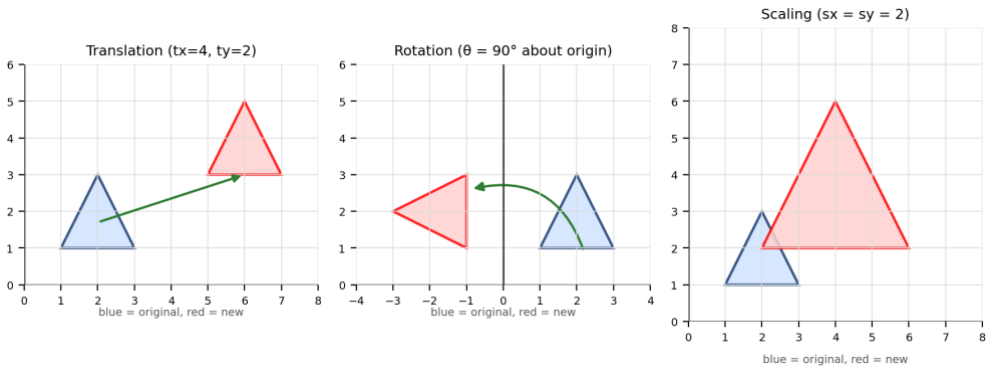


Fig. 1: Translation, rotation and scaling

Transformation	What it does	Equation
Translation	Moves object along a straight path.	$x' = x + tx,\ \ y' = y + ty$
Rotation	Turns object by angle θ about a point.	$x' = x\cos\theta - y\sin\theta,\ \ y' = x\sin\theta + y\cos\theta$
Scaling	Makes object bigger or smaller.	$x' = x \cdot sx,\ \ y' = y \cdot sy$
Reflection	Gives mirror image about an axis.	About x-axis: $x' = x,\ \ y' = -y$
Shear	Slants the object.	x-shear: $x' = x + shx \cdot y,\ \ y' = y$

Q2. Translation, Rotation and Scaling in 2D (Matrix Form)

1. Translation

- Moves every point by **tx** along x and **ty** along y. Shape and size do not change.

$$P' = P + T$$

$$\begin{vmatrix} x' & = & x & + & tx \\ y' & = & y & + & ty \end{vmatrix}$$

2. Rotation (about origin, anticlockwise θ)

- Positive θ = anticlockwise rotation. Distance from the origin stays the same.

$$P' = R \cdot P$$

$$R(\theta) = \begin{vmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{vmatrix}$$

3. Scaling (about origin)

- sx, sy > 1 enlarge; 0 < sx, sy < 1 shrink. If sx = sy it is **uniform scaling**.

```
P' = S · P
```

$$S(sx, sy) = \begin{vmatrix} sx & 0 \\ 0 & sy \end{vmatrix}$$

Problem: translation is an **addition** while rotation and scaling are **multiplications**. So they cannot be combined easily. Homogeneous coordinates (Q3) solve this.

Q3. Homogeneous Coordinates

Homogeneous coordinates represent a 2D point (x, y) by **three** values (xh, yh, h), where $x = xh / h$ and $y = yh / h$.

- Normally we take **h = 1**, so the point is written as **(x, y, 1)**.
- Example: (2, 3) → (2, 3, 1). Also (4, 6, 2) is the same point (2, 3).

Why are they used?

- **Translation becomes a matrix multiplication**, like rotation and scaling.
- All transformations use the **same 3 × 3 matrix form**, so one common method works for all.
- Many transformations can be **combined (multiplied) into one matrix**.
- Makes hardware and software simple and fast.
- Needed for **perspective** projection in 3D and for points at infinity (h = 0).

Q4. 2D Transformations Using Homogeneous Matrices

Write the point as a column $\mathbf{P = [x\ y\ 1]^T}$ and use $\mathbf{P' = M \cdot P}$.

Translation T(tx, ty)

$$\begin{vmatrix} 1 & 0 & tx \\ 0 & 1 & ty \\ 0 & 0 & 1 \end{vmatrix}$$

Rotation R(θ)

$$\begin{vmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{vmatrix}$$

Scaling S(sx, sy)

$$\begin{vmatrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & 1 \end{vmatrix}$$

Inverse matrices

- $T^{-1}(tx, ty) = T(-tx, -ty)$
- $R^{-1}(\theta) = R(-\theta)$
- $S^{-1}(sx, sy) = S(1/sx, 1/sy)$

Reflection about x-axis and shear (x-direction)

$$\begin{vmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{vmatrix}$$

$$\begin{vmatrix} 1 & shx & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{vmatrix}$$

Q5. Composition of Transformations

Composition (concatenation) means applying two or more transformations one after another. Their matrices are **multiplied** to form a **single composite matrix**.

```
P' = Mn · … · M2 · M1 · P        →        M = Mn · … · M2 · M1
```

- The first transformation is the **rightmost** matrix.
- Matrix multiplication is **not commutative** ($M2 \cdot M1 \neq M1 \cdot M2$), so the **order matters**.

Steps to combine

1. Break the work into basic transformations M1, M2, … Mn.
2. Write their matrices in the order of use.
3. Multiply them from **right to left** to get M.
4. Multiply M with every point of the object (only one multiplication per point).

Useful rules

- Two translations: $T(a,b) \cdot T(c,d) = T(a+c, b+d)$
- Two rotations: $R(\theta 1) \cdot R(\theta 2) = R(\theta 1 + \theta 2)$
- Two scalings: $S(a,b) \cdot S(c,d) = S(a \cdot c, b \cdot d)$

Example: Rotation about a pivot point (xr, yr)

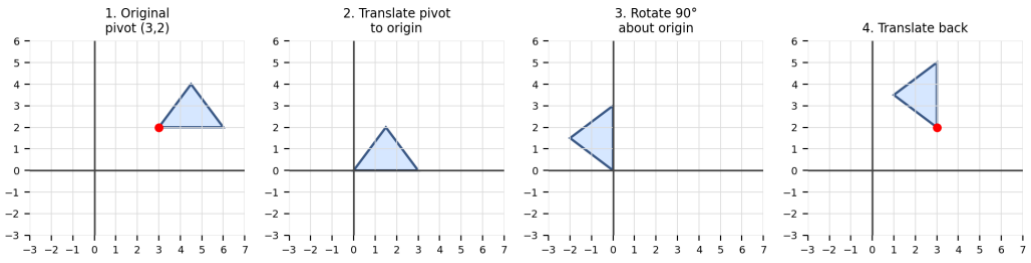


Fig. 2: Rotation about a pivot point

1. Translate pivot to origin: $T(-xr, -yr)$
2. Rotate about origin: $R(\theta)$
3. Translate back: $T(xr, yr)$

```
M = T(xr, yr) · R(θ) · T(-xr, -yr)
```

In the same way, **scaling about a fixed point** uses $M = T(xf, yf) \cdot S(sx, sy) \cdot T(-xf, -yf)$.

Q6. Numerical: Translate P(2, 3) by (4, 5), then Rotate 90°

Method 1: Step by step

- Translation: $x' = 2 + 4 = \mathbf{6}$, $y' = 3 + 5 = \mathbf{8} \rightarrow P' = (6, 8)$
- Rotation by 90° (anticlockwise): $\cos 90° = 0$, $\sin 90° = 1$
- $x'' = x'\cos\theta - y'\sin\theta = 6(0) - 8(1) = \mathbf{-8}$
- $y'' = x'\sin\theta + y'\cos\theta = 6(1) + 8(0) = \mathbf{6}$

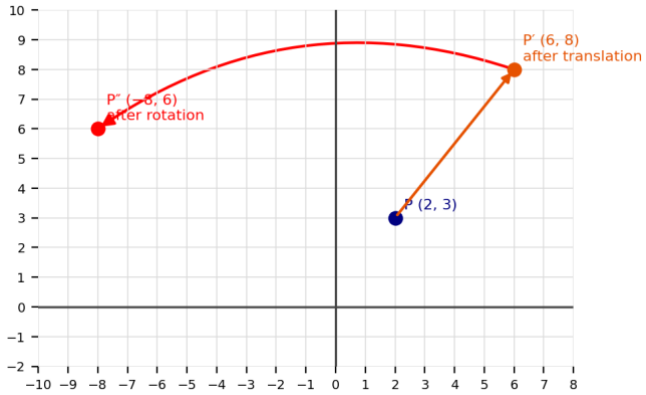


Fig. 3: Path of the point

Method 2: Using composite matrix

$$\begin{vmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{vmatrix}$$

$$\times T =$$

$$\begin{vmatrix} 1 & 0 & 4 \\ 0 & 1 & 5 \\ 0 & 0 & 1 \end{vmatrix}$$

$$\text{Composite } M = R \cdot T =$$

$$\begin{vmatrix} 0 & -1 & -5 \\ 1 & 0 & 4 \\ 0 & 0 & 1 \end{vmatrix}$$

$$x'' = 0(2) - 1(3) - 5 = -8 \;, \quad y'' = 1(2) + 0(3) + 4 = 6$$

Final coordinates: P″ = (−8, 6)

Q7. Window-to-Viewport Transformation

A **window** is the area of the world (real) coordinate system that we want to display. A **viewport** is the area on the screen where it is shown. The **window-to-viewport transformation** maps each point of the window to the matching point in the viewport, so the picture is shown at the right position and size.

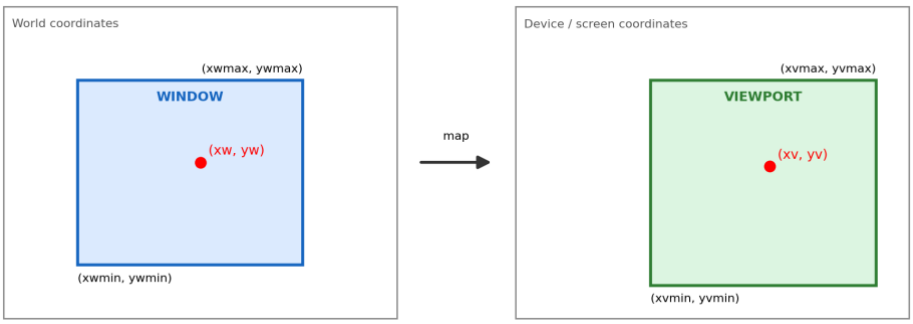


Fig. 4: Window to viewport mapping

Derivation

The relative position of the point inside the window must be the same as inside the viewport:

```
(xw − xwmin) / (xwmax − xwmin) = (xv − xvmin) / (xvmax − xvmin)
(yw − ywmin) / (ywmax − ywmin) = (yv − yvmin) / (yvmax − yvmin)
```

Solving for xv and yv:

```
xv = xvmin + (xw − xwmin) · sx
yv = yvmin + (yw − ywmin) · sy
sx = (xvmax − xvmin) / (xwmax − xwmin)
sy = (yvmax − yvmin) / (ywmax − ywmin)
```

- If sx = sy the picture keeps its shape; if sx ≠ sy it gets **stretched**.

Steps (as transformations)

1. Translate window corner to origin: T(−xwmin, −ywmin)
2. Scale window to viewport size: S(sx, sy)
3. Translate to viewport position: T(xvmin, yvmin)

Example

Window (20,20)–(100,80), viewport (0,0)–(400,300). Map point (60, 50).

- sx = 400 / 80 = **5**, sy = 300 / 60 = **5**
- xv = 0 + (60 − 20) × 5 = **200**, yv = 0 + (50 − 20) × 5 = **150**
- Viewport point = **(200, 150)**

Q8. Efficiency in Geometrical Transformations

Efficiency means getting the transformed picture using the **least calculations (time and memory)**. This is important because a picture has thousands of points and must be redrawn quickly (e.g. in animation).

How matrix composition improves efficiency

- Without composition, every point goes through **each** transformation one by one.
- With composition, the matrices are multiplied **once** and every point is transformed by the **single** matrix.
- Fewer multiplications and additions per point → **faster** display.
- No need to store results of intermediate steps → **less memory**, and less rounding error.
- Matrix methods suit **hardware** (graphics cards) that do matrix work very fast.

Example

Apply 3 transformations to n points using 3 × 3 matrices (9 multiplications per point-matrix product):

Method	Multiplications
Separately (3 matrices per point)	3 × 9 × n = 27n
Composite (build M once, then use)	2 matrix products (2 × 27 = 54) + 9n

For n = 100 points: 2700 vs 954 multiplications. The bigger the object, the greater the saving.

- Further saving: the last row of the matrix is always (0, 0, 1), so it need not be calculated.
- Efficiency tips: use **inverse matrices** for undo; rotate only **vertices**, not every pixel; use standard values of sin/cos from a table.

Q9. 3D Transformations (Matrix Representation)

In 3D a point is (x, y, z). Using homogeneous coordinates it becomes **(x, y, z, 1)**, so all transformations are **4 × 4 matrices** and **P′ = M · P**. We use a **right-handed** system.

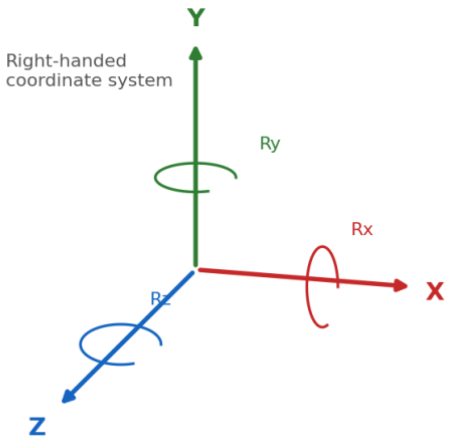


Fig. 5: Right-handed 3D axes and rotations

1. Translation T(tx, ty, tz)

$$\begin{vmatrix} 1 & 0 & 0 & tx \\ 0 & 1 & 0 & ty \\ 0 & 0 & 1 & tz \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

$$x' = x + tx,\ \ y' = y + ty,\ \ z' = z + tz$$

2. Scaling S(sx, sy, sz)

$$\begin{vmatrix} sx & 0 & 0 & 0 \\ 0 & sy & 0 & 0 \\ 0 & 0 & sz & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

$$x' = x \cdot sx,\ \ y' = y \cdot sy,\ \ z' = z \cdot sz$$

3. Rotation

- 3D rotation is done about an **axis**. Positive angle = anticlockwise when looking from the positive axis towards the origin.

About Z-axis Rz(θ) (z does not change)

$$\begin{vmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

About X-axis Rx(θ) (x does not change)

$$\begin{vmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

About Y-axis Ry(θ) (y does not change)

$$\begin{vmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

- Rotation about any other axis = translate axis to origin, rotate, translate back (composition).

Q10. Transformation as a Change of Coordinate System

There are two ways to look at a transformation:

- **Object transformation** – the object moves; the coordinate system stays fixed.
- **Coordinate-system transformation** – the object stays fixed; the axes are moved, so the same point gets **new coordinates**.

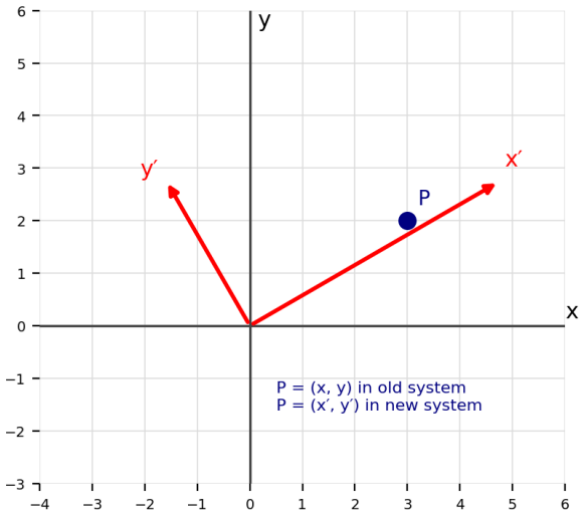


Fig. 6: Same point in old and new axes

Both give the same result but the matrix of the coordinate change is the **inverse** of the object transformation.

Coordinate system changes

Change of axes	New coordinates
Translate origin to (x0, y0)	$x' = x - x0,\ \ y' = y - y0$
Rotate axes by angle θ	$x' = x\cos\theta + y\sin\theta,\ \ y' = -x\sin\theta + y\cos\theta$
Scale axes by sx, sy	$x' = x / sx,\ \ y' = y / sy$

Matrix for axes translation: $T(-x0, -y0)$. Matrix for axes rotation: $R(-\theta)$.

Difference

Object transformation	Coordinate-system transformation
Object moves, axes fixed.	Axes move, object fixed.
Point gets new position in same system.	Same point gets new coordinates in new system.
Uses matrix M, e.g. T(tx, ty).	Uses inverse matrix M^{-1}, e.g. T(−tx, −ty).
Moving object by +5 in x.	Moving axes by −5 in x (same result).
Used for animation and movement.	Used for window-viewport, viewing and local-to-world mapping.